

2B/3B Döndürme Matematiği ve Unity'nin Yaklaşımı

Rotasyon matrislerinden quaternion'a: temsil, sıra ve motorlar arası farklar.

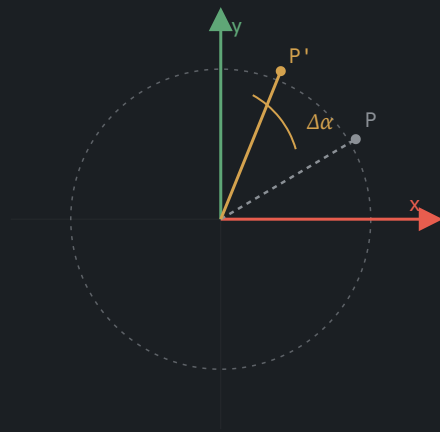


2 Boyutlu Düzlemde Döndürme

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \Delta\alpha & -\sin \Delta\alpha \\ \sin \Delta\alpha & \cos \Delta\alpha \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Noktayı orijin etrafında $\Delta\alpha$ kadar döndüren 2×2 rotasyon matrisi.

$$r = \sqrt{x^2 + y^2} \text{ her zaman korunur}$$

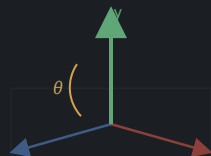


3 Boyutta Tek Eksen Etrafında Döndürme

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$



$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$



$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



$$\vec{v}' = R(\theta) \vec{v}$$

İki ve Üç Eksende Eşzamanlı Döndürme: Sıra Önemlidir

Rotasyon matrisleri **komütatif değildir**:

$$R_a R_b \neq R_b R_a$$

"Aynı anda" birden fazla eksende döndürme, motorun belirli bir sabit sırayla art arda uyguladığı döndürmelerdir:

$$\vec{v}'_{3 \text{ eksen, Unity}} = R_y(\theta_y) R_x(\theta_x) R_z(\theta_z) \vec{v}$$

Sayısal kanıt: aynı iki açığı (X'te 30°, Y'de 45°) farklı sırayla uygulamak *iki farklı* sonuç verir. Sayısal kanıt bir sonraki slaytta.

Sayısal Kanıt

Başlangıç vektörü: $\vec{v}_0 = (1, 0.5, -0.3)$

SIRA A · önce Y(45°), sonra X(30°)

$$\begin{aligned} R_y(45^\circ)\vec{v}_0 &= \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 1 & 0 \\ -0.7071 & 0 & 0.7071 \end{bmatrix} \begin{bmatrix} 1 \\ 0.5 \\ -0.3 \end{bmatrix} \\ &= \begin{bmatrix} 0.7071(1) + 0(0.5) + 0.7071(-0.3) \\ 0(1) + 1(0.5) + 0(-0.3) \\ -0.7071(1) + 0(0.5) + 0.7071(-0.3) \end{bmatrix} = \begin{bmatrix} 0.4950 \\ 0.5000 \\ -0.9192 \end{bmatrix} \\ R_x(30^\circ)\vec{v}_1 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.8660 & -0.5 \\ 0 & 0.5 & 0.8660 \end{bmatrix} \begin{bmatrix} 0.4950 \\ 0.5000 \\ -0.9192 \end{bmatrix} \\ &= \begin{bmatrix} 0.4950 \\ 0.8660(0.5000) - 0.5(-0.9192) \\ 0.5(0.5000) + 0.8660(-0.9192) \end{bmatrix} = \begin{bmatrix} 0.4950 \\ 0.8926 \\ -0.5460 \end{bmatrix} \end{aligned}$$

Sonuç A = (0.4950, 0.8926, -0.5460)

SIRA B · önce X(30°), sonra Y(45°)

$$\begin{aligned} R_x(30^\circ)\vec{v}_0 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.8660 & -0.5 \\ 0 & 0.5 & 0.8660 \end{bmatrix} \begin{bmatrix} 1 \\ 0.5 \\ -0.3 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 0.8660(0.5) - 0.5(-0.3) \\ 0.5(0.5) + 0.8660(-0.3) \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.5830 \\ -0.0098 \end{bmatrix} \\ R_y(45^\circ)\vec{v}_1 &= \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 1 & 0 \\ -0.7071 & 0 & 0.7071 \end{bmatrix} \begin{bmatrix} 1.0000 \\ 0.5830 \\ -0.0098 \end{bmatrix} \\ &= \begin{bmatrix} 0.7071(1) + 0.7071(-0.0098) \\ 0.5830 \\ -0.7071(1) + 0.7071(-0.0098) \end{bmatrix} = \begin{bmatrix} 0.7002 \\ 0.5830 \\ -0.7140 \end{bmatrix} \end{aligned}$$

Sonuç B = (0.7002, 0.5830, -0.7140)

Güncel Standart: Quaternion

$$q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right) (u_x i + u_y j + u_z k)$$

$$\vec{v}' = q \vec{v} q^{-1}$$

Hamilton tarafından 16 Ekim 1843'te keşfedildi; bilgisayar grafiklerinde yaygınlaşması Shoemake'in 1985 SIGGRAPH bildirisiyle oldu.

Quaternion

Gimbal kilit yok · 4 sayı · ucuz normalizasyon (tek karekök) · pürüzsüz interpolasyon

Euler açıları

Yalnızca gösterim · gimbal kilide takılır

Rotasyon matrisi (3×3)

9 sayı, gereksiz fazlalık · tekrarlı çarpımda sürüklenir

Hamilton Çarpımı, Sayısal Örnek

$q_1 = 90^\circ$ X eksenini $q_2 = 90^\circ$ Y eksenini

$$q_1 = (0.7071, 0.7071, 0, 0), \quad q_2 = (0.7071, 0, 0.7071, 0)$$

$$q_1 q_2 = (w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2, w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2, w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2, w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2)$$

$$w = 0.7071(0.7071) - 0.7071(0) - 0(0.7071) - 0(0) = 0.5000$$

$$x = 0.7071(0) + 0.7071(0.7071) + 0(0) - 0(0.7071) = 0.5000$$

$$y = 0.7071(0.7071) - 0.7071(0) + 0(0.7071) + 0(0) = 0.5000$$

$$z = 0.7071(0) + 0.7071(0.7071) - 0(0) + 0(0.7071) = 0.5000$$

$$q_1 \cdot q_2 = (0.5, 0.5, 0.5, 0.5)$$

Sürüklenme (drift) karşılaştırması

5000 ardışık küçük rastgele döndürme, float32 hassasiyetle uygulandığında:

Matris ortogonalite hatası

$$\|R^T R - I\| = 1.75 \times 10^{-5}$$

Quaternion norm hatası

$$||q| - 1| = 8.98 \times 10^{-5}$$

Düzeltilme maliyeti farklı: quaternion $\rightarrow q/|q|$ (tek karekök);
matris $\rightarrow$ tam Gram-Schmidt yeniden ortogonalleştirme.

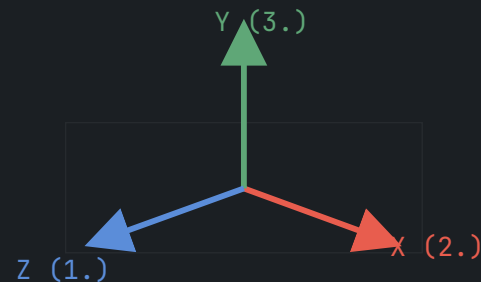
Unity'nin Yaklaşımı: İçeride Quaternion, Dışarıda Euler

Unity, her rotasyonu içeride **quaternion** olarak tutar.

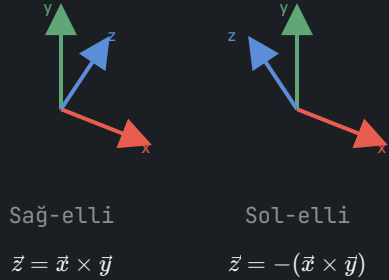
Inspector'daki X, Y, Z açıları yalnızca bir görüntüleme kolaylığıdır.

Üç eksen birden ayarlandığında Unity resmi olarak **Z → X → Y** sırasını uygular:

$$\vec{v}' = R_y(\theta_y) R_x(\theta_x) R_z(\theta_z) \vec{v} = R(q_y q_x q_z) \vec{v}$$



Sağ-el / Sol-el Kuralı ve Yukarı Eksen



Motor / API	El Kuralı	Yukarı
Unity	Sol-elli	Y
Unreal Engine	Sol-elli	Z
Godot	Sağ-elli	Y
OpenGL	Sağ-elli	Y
DirectX	Sol-elli	Y

Kaynak: Unity [1] · Unreal Engine 5 Docs [4] · Godot forum [5] · LearnOpenGL [6] · Microsoft Learn [7]

Özet

- 01 2B'de tek parametrelili ($\Delta\alpha$) rotasyon matrisi yeterlidir.
- 02 3B'de tek/çift/üç eksen döndürme, aynı R_x , R_y , R_z matrislerinin belirli bir sırada çarpımıdır; sıra sonucu değiştirir.
- 03 Quaternion, gimbal kilit yaşamaması, kompaktlığı ve ucuz normalizasyonu sayesinde günümüzün fiilî standardıdır.
- 04 Unity içeride quaternion kullanır, dışarıda $Z \rightarrow X \rightarrow Y$ sıralı Euler açılarını gösterir.
- 05 Motorlar arasında sağ-el/sol-el ve yukarı eksen farkı vardır; formülleri taşırken dikkat edilmesi gereken en önemli nokta budur.